보안 계층 예제 코드

개요

이 문서는 Rust libp2p의 보안 계층 구현을 실제 코드 예제를 통해 설명합니다. Noise 프로토콜과 TLS 구현, 피어 인증, 키 교환 메커니즘을 포함한 완전한 예제를 제공합니다.

1. 기본 Noise 프로토콜 설정 예제

1.1 Noise 설정 및 키 생성

```rust
use libp2p_core::upgrade::Version;

use libp2p_identity as identity;

use libp2p_noise::{Config as NoiseConfig, Error as NoiseError};

use std::error::Error;


/// Noise 프로토콜을 위한 기본 설정을 생성하는 예제

pub  fn create_noise_config() -> Result<NoiseConfig, Box<dyn Error>> {

// 1. Identity 키페어 생성 (Ed25519 권장)

let identity_keypair = identity::Keypair::generate_ed25519();

// 2. Noise 설정 생성

let noise_config = NoiseConfig::new(&identity_keypair)?;

println!("✅ Noise 설정 생성 완료");
```

```
println!(" - Identity 키 타입: {:?}", identity_keypair.key_type());
println!(" - PeerId: {}", identity_keypair.public().to_peer_id());
Ok(noise_config)
}

/// 사용자 정의 프롤로그를 포함한 Noise 설정
pub  fn create_noise_config_with_prologue(
prologue: Vec<u8>
) -> Result<NoiseConfig, Box<dyn Error>> {
let identity_keypair = identity::Keypair::generate_ed25519();
let noise_config = NoiseConfig::new(&identity_keypair)?
.with_prologue(prologue);
println!("✅ 프롤로그 포함 Noise 설정 생성 완료");
Ok(noise_config)
}
```

1.2 WebTransport 지원 Noise 설정

```
use libp2p_core::multihash::{Multihash, Code};
use std::collections::HashSet;

/// WebTransport 인증서 해시를 포함한 Noise 설정
pub fn create_noise_config_with_webtransport(
```

```
cert_hashes: Vec<Vec<u8>>

) -> Result<NoiseConfig, Box<dyn Error>> {

let identity_keypair = identity::Keypair::generate_ed25519();

// 인증서 해시를 Multihash로 변환

let mut certhashes = HashSet::new();

for hash in cert_hashes {

let multihash = Multihash::wrap(Code::Sha256.into(), &hash)

.map_err(|e| format!("Invalid certificate hash: {}", e))?;

certhashes.insert(multihash);

}

let noise_config = NoiseConfig::new(&identity_keypair)?

.with_webtransport_certhashes(certhashes);

println!("✅ WebTransport 지원 Noise 설정 생성 완료");

println!(" - 인증서 해시 개수: {}", certhashes.len());

Ok(noise_config)

}
```

2. 완전한 Noise 핸드셰이크 예제

2.1 Initiator (연결 시작자) 구현

```
use libp2p_core::{

transport::{Transport, ListenerId},
```

```
multiaddr::Multiaddr,

PeerId,

};

use libp2p_tcp as tcp;

use libp2p_noise as noise;

use futures::prelude::*;

use std::error::Error;



/// Noise 프로토콜을 사용한 연결 시작자 예제

pub async fn noise_initiator_example(

target_addr: Multiaddr,

expected_peer_id: Option<PeerId>

) -> Result<(), Box<dyn Error>> {

// 1. Identity 키페어 생성

let local_keypair = identity::Keypair::generate_ed25519();

let local_peer_id = local_keypair.public().to_peer_id();

println!("🚀 Noise Initiator 시작");

println!(" - Local PeerId: {}", local_peer_id);

println!(" - Target Address: {}", target_addr);

// 2. Transport 설정

let tcp_transport = tcp::tokio::Transport::new(tcp::Config::default());

let noise_config = noise::Config::new(&local_keypair)?;

// 3. Transport 업그레이드 체인 구성
```

```
let transport = tcp_transport
.upgrade(Version::V1)
.authenticate(noise_config)
.multiplex(libp2p_yamux::Config::default());
// 4. 연결 시도
println!("🔗 연결 시도 중...");
let (remote_peer_id, connection) = transport
.dial(target_addr.clone())?
.await?;
// 5. 피어 검증
if let Some(expected) = expected_peer_id {
if remote_peer_id != expected {
return Err(format!(
"예상된 PeerId({})와 실제 PeerId({})가 다릅니다",
expected, remote_peer_id
).into());
}
}
println!("✅ Noise 핸드셰이크 성공!");
println!(" - Remote PeerId: {}", remote_peer_id);
println!(" - 연결 설정 완료");
// 6. 연결 사용 (예: 간단한 데이터 전송)
// connection을 사용하여 실제 통신 수행
```

```
Ok(())

}
```

2.2 Responder (응답자) 구현

```
use libp2p_core::transport::ListenerEvent;

use tokio::net::TcpListener;


/// Noise 프로토콜을 사용한 응답자 예제

pub async fn noise_responder_example(

listen_addr: Multiaddr

) -> Result<(), Box<dyn Error>> {

// 1. Identity 키페어 생성

let local_keypair = identity::Keypair::generate_ed25519();

let local_peer_id = local_keypair.public().to_peer_id();

println!("🎯 Noise Responder 시작");

println!(" - Local PeerId: {}", local_peer_id);

println!(" - Listen Address: {}", listen_addr);

// 2. Transport 설정

let tcp_transport = tcp::tokio::Transport::new(tcp::Config::default());

let noise_config = noise::Config::new(&local_keypair)?;

// 3. Transport 업그레이드 체인 구성

let mut transport = tcp_transport
```

```
.upgrade(Version::V1)
.authenticate(noise_config)
.multiplex(libp2p_yamux::Config::default());
// 4. 리스너 시작
let listener_id = transport.listen_on(listen_addr.clone())?;
println!("👂 리스닝 시작됨 (ID: {:?})", listener_id);
// 5. 연결 수락 루프
loop {
match transport.select_next_some().await {
ListenerEvent::NewAddress { address, .. } => {
println!("📍 새로운 리스닝 주소: {}", address);
}
ListenerEvent::Upgrade { upgrade, .. } => {
println!("🔄 새로운 연결 업그레이드 중...");
match upgrade.await {
Ok((remote_peer_id, connection)) => {
println!("✅ Noise 핸드셰이크 성공!");
println!(" - Remote PeerId: {}", remote_peer_id);
// 연결 처리 (별도 태스크에서)
tokio::spawn(async move {
handle_connection(remote_peer_id, connection).await;
});
}
```

```
Err(e) => {
println!("❌ 핸드셰이크 실패: {}", e);
}
}
}
ListenerEvent::Error { error } => {
println!("❌ 리스너 오류: {}", error);
}
_ => {}
}
}
}


/// 연결 처리 함수
async fn handle_connection(
peer_id: PeerId,
mut connection: impl AsyncRead + AsyncWrite + Unpin
) {
println!("🔄 연결 처리 시작: {}", peer_id);
// 실제 프로토콜 처리 로직
// 예: 데이터 읽기/쓰기, 프로토콜 네고시에이션 등
println!("✅ 연결 처리 완료: {}", peer_id);
```

```
}
```

3. TLS 보안 계층 예제

3.1 TLS 설정 및 인증서 생성

```
use libp2p_tls as tls;

use rustls::{ClientConfig, ServerConfig};


/// TLS 클라이언트 설정 생성 예제

pub fn create_tls_client_config(

local_keypair: &identity::Keypair,

expected_peer_id: Option<PeerId>

) -> Result<ClientConfig, Box<dyn Error>> {

println!("🔐 TLS 클라이언트 설정 생성 중...");

let config = tls::make_client_config(local_keypair, expected_peer_id)?;

println!("✅ TLS 클라이언트 설정 완료");

if let Some(peer_id) = expected_peer_id {

println!(" - 예상 PeerId: {}", peer_id);

}

Ok(config)

}
```

```rust
/// TLS 서버 설정 생성 예제
pub fn create_tls_server_config(
local_keypair: &identity::Keypair
) -> Result<ServerConfig, Box<dyn Error>> {
println!("🔐 TLS 서버 설정 생성 중...");
let config = tls::make_server_config(local_keypair)?;
println!("✅ TLS 서버 설정 완료");
println!(" - Local PeerId: {}", local_keypair.public().to_peer_id());
Ok(config)
}
```

3.2 TLS 인증서 검증 예제

```rust
use libp2p_tls::certificate;

/// libp2p TLS 인증서 생성 및 검증 예제
pub fn tls_certificate_example() -> Result<(), Box<dyn Error>> {
// 1. Identity 키페어 생성
let identity_keypair = identity::Keypair::generate_ed25519();
let peer_id = identity_keypair.public().to_peer_id();
println!("📜 TLS 인증서 예제");
println!(" - PeerId: {}", peer_id);
// 2. 인증서 생성
```

```
let (certificate, private_key) = certificate::generate(&identity_keypair)?;

println!("✅ 인증서 생성 완료");

println!(" - 인증서 크기: {} bytes", certificate.as_ref().len());

println!(" - 개인키 크기: {} bytes", private_key.secret_der().len());

// 3. 인증서에서 PeerId 추출 및 검증

let extracted_peer_id = certificate::parse_certificate(&certificate)?;

if extracted_peer_id == peer_id {

println!("✅ 인증서 검증 성공!");

println!(" - 추출된 PeerId: {}", extracted_peer_id);

} else {

return Err("인증서 검증 실패: PeerId 불일치".into());

}

Ok(())

}
```

4. 키 교환 및 암호화 예제

4.1 X25519 키 교환 예제

```
use libp2p_noise::x25519_spec::{Keypair as X25519Keypair, PublicKey, SecretKey};

use rand::RngCore;


/// X25519 키 교환 시뮬레이션 예제
```

```rust
pub fn x25519_key_exchange_example() -> Result<(), Box<dyn Error>> {
println!("🔑 X25519 키 교환 예제");
// 1. Alice의 키페어 생성
let alice_keypair = X25519Keypair::new();
println!(" - Alice 공개키: {:?}", alice_keypair.public());
// 2. Bob의 키페어 생성
let bob_keypair = X25519Keypair::new();
println!(" - Bob 공개키: {:?}", bob_keypair.public());
// 3. 공유 비밀 계산 (Alice 측)
let mut alice_shared_secret = [0u8; 32];
alice_keypair.dh(bob_keypair.public().as_ref(), &mut alice_shared_secret)
.map_err(|e| format!("Alice DH 실패: {:?}", e))?;
// 4. 공유 비밀 계산 (Bob 측)
let mut bob_shared_secret = [0u8; 32];
bob_keypair.dh(alice_keypair.public().as_ref(), &mut bob_shared_secret)
.map_err(|e| format!("Bob DH 실패: {:?}", e))?;
// 5. 공유 비밀 검증
if alice_shared_secret == bob_shared_secret {
println!("✅ 키 교환 성공!");
println!(" - 공유 비밀: {:02x?}", &alice_shared_secret[..8]);
} else {
return Err("키 교환 실패: 공유 비밀 불일치".into());
}
```

```
Ok(())

}
```

4.2 Ed25519 서명 및 검증 예제

```
/// Ed25519 서명 생성 및 검증 예제

pub fn ed25519_signature_example() -> Result<(), Box<dyn Error>> {

println!("✍️ Ed25519 서명 예제");

// 1. 키페어 생성

let keypair = identity::Keypair::generate_ed25519();

let public_key = keypair.public();

println!(" - 공개키: {}", public_key.to_peer_id());

// 2. 서명할 메시지

let message = b"Hello, libp2p security layer!";

println!(" - 메시지: {:?}", std::str::from_utf8(message)?);

// 3. 서명 생성

let signature = keypair.sign(message)?;

println!(" - 서명 크기: {} bytes", signature.len());

// 4. 서명 검증

let is_valid = public_key.verify(message, &signature);

if is_valid {

println!("✅ 서명 검증 성공!");

} else {
```

```
return Err("서명 검증 실패".into());
}
// 5. 잘못된 메시지로 검증 (실패해야 함)
let wrong_message = b"Wrong message";
let is_invalid = public_key.verify(wrong_message, &signature);
if !is_invalid {
println!("✅ 잘못된 서명 검증 정상적으로 실패");
} else {
return Err("보안 오류: 잘못된 서명이 검증됨".into());
}
Ok(())
}
```

5. 피어 인증 시뮬레이션 예제

5.1 완전한 피어 인증 과정

```
use libp2p_noise::protocol::{AuthenticKeypair, KeypairIdentity};

/// 피어 간 상호 인증 시뮬레이션
pub fn peer_authentication_simulation() -> Result<(), Box<dyn Error>> {
println!("🤝 피어 인증 시뮬레이션");
// 1. Alice와 Bob의 Identity 키페어 생성
```

```
let alice_identity = identity::Keypair::generate_ed25519();

let bob_identity = identity::Keypair::generate_ed25519();

let alice_peer_id = alice_identity.public().to_peer_id();

let bob_peer_id = bob_identity.public().to_peer_id();

println!(" - Alice PeerId: {}", alice_peer_id);

println!(" - Bob PeerId: {}", bob_peer_id);

// 2. 각자의 DH 키페어 생성

let alice_dh_keypair = X25519Keypair::new();

let bob_dh_keypair = X25519Keypair::new();

// 3. DH 키페어를 Identity 키로 인증

let alice_authentic = alice_dh_keypair.into_authentic(&alice_identity)?;

let bob_authentic = bob_dh_keypair.into_authentic(&bob_identity)?;

println!("✅ DH 키페어 인증 완료");

// 4. 서명 검증 시뮬레이션 (Alice가 Bob의 서명 검증)

let bob_dh_pubkey = bob_authentic.keypair.public();

let bob_signature = &bob_authentic.identity.signature;

let bob_identity_pubkey = &bob_authentic.identity.public;

// 서명 검증용 메시지 구성

let verification_message = [

b"noise-libp2p-static-key:".as_slice(),

bob_dh_pubkey.as_ref()

].concat();

let is_bob_valid = bob_identity_pubkey.verify(&verification_message, bob_signatu
```

```
if is_bob_valid {
println!("✅ Alice가 Bob의 서명 검증 성공");
} else {
return Err("Bob의 서명 검증 실패".into());
}
// 5. 반대 방향 검증 (Bob이 Alice의 서명 검증)
let alice_dh_pubkey = alice_authentic.keypair.public();
let alice_signature = &alice_authentic.identity.signature;
let alice_identity_pubkey = &alice_authentic.identity.public;
let verification_message = [
b"noise-libp2p-static-key:".as_slice(),
alice_dh_pubkey.as_ref()
].concat();
let is_alice_valid = alice_identity_pubkey.verify(&verification_message, alice_s
if is_alice_valid {
println!("✅ Bob이 Alice의 서명 검증 성공");
} else {
return Err("Alice의 서명 검증 실패".into());
}
println!("🎉 상호 인증 완료!");
Ok(())
}
```

6. 보안 계층 통합 예제

6.1 완전한 보안 Transport 구성

```rust
use libp2p_core::Transport;
use libp2p_tcp as tcp;
use libp2p_noise as noise;
use libp2p_yamux as yamux;

/// 완전한 보안 Transport 스택 구성 예제
pub fn create_secure_transport() -> Result<impl Transport<Output = (PeerId, impl
println!("🔒 보안 Transport 스택 구성");
// 1. Identity 키페어 생성
let identity_keypair = identity::Keypair::generate_ed25519();
let local_peer_id = identity_keypair.public().to_peer_id();
println!(" - Local PeerId: {}", local_peer_id);
// 2. 기본 TCP Transport
let tcp_transport = tcp::tokio::Transport::new(tcp::Config::default());
println!(" ✅ TCP Transport 설정");
// 3. Noise 보안 계층
let noise_config = noise::Config::new(&identity_keypair)?;
println!(" ✅ Noise 보안 계층 설정");
// 4. Yamux 멀티플렉싱 계층
```

```
let yamux_config = yamux::Config::default();

println!(" ✅ Yamux 멀티플렉싱 설정");

// 5. Transport 체인 구성

let transport = tcp_transport

.upgrade(Version::V1)

.authenticate(noise_config)

.multiplex(yamux_config);

println!("🎯 보안 Transport 스택 완성!");

println!(" - 계층: TCP → Noise → Yamux");

Ok(transport)

}
```

6.2 보안 설정 검증 예제

```
/// 보안 설정의 유효성을 검증하는 예제

pub fn validate_security_configuration() -> Result<(), Box<dyn Error>> {

println!("🔍 보안 설정 검증");

// 1. 다양한 키 타입 테스트

let key_types = [

("Ed25519", || identity::Keypair::generate_ed25519()),

("RSA", || identity::Keypair::generate_rsa(2048)),

("Secp256k1", || identity::Keypair::generate_secp256k1()),

];
```

```
for (name, generator) in key_types.iter() {
println!(" 🔑 {} 키 테스트", name);
let keypair = generator();
let peer_id = keypair.public().to_peer_id();
// Noise 설정 생성 테스트
match noise::Config::new(&keypair) {
Ok(_) => println!(" ✅ Noise 설정 성공"),
Err(e) => println!(" ❌ Noise 설정 실패: {}", e),
}
// TLS 설정 생성 테스트
match tls::make_client_config(&keypair, None) {
Ok(_) => println!(" ✅ TLS 클라이언트 설정 성공"),
Err(e) => println!(" ❌ TLS 클라이언트 설정 실패: {}", e),
}
match tls::make_server_config(&keypair) {
Ok(_) => println!(" ✅ TLS 서버 설정 성공"),
Err(e) => println!(" ❌ TLS 서버 설정 실패: {}", e),
}
println!(" 📋 PeerId: {}", peer_id);
}
println!("✅ 보안 설정 검증 완료");
Ok(())
```

```
}
```

7. 실행 가능한 메인 함수

7.1 모든 예제를 실행하는 메인 함수

```
#[tokio::main]

async fn main() -> Result<(), Box<dyn Error>> {

println!("🚀 libp2p 보안 계층 예제 실행");

println!("=" .repeat(50));

// 1. 기본 Noise 설정

println!("\n1️⃣ Noise 설정 예제");

create_noise_config()?;

// 2. 키 교환 예제

println!("\n2️⃣ X25519 키 교환 예제");

x25519_key_exchange_example()?;

// 3. 서명 예제

println!("\n3️⃣ Ed25519 서명 예제");

ed25519_signature_example()?;

// 4. TLS 인증서 예제

println!("\n4️⃣ TLS 인증서 예제");

tls_certificate_example()?;

// 5. 피어 인증 시뮬레이션
```

```
println!("\n5️⃣ 피어 인증 시뮬레이션");
peer_authentication_simulation()?;
// 6. 보안 Transport 구성
println!("\n6️⃣ 보안 Transport 구성");
let _transport = create_secure_transport()?;
// 7. 보안 설정 검증
println!("\n7️⃣ 보안 설정 검증");
validate_security_configuration()?;
println!("\n🎉 모든 예제 실행 완료!");
Ok(())
}
```

8. Cargo.toml 의존성

```
[dependencies]
libp2p-core = "0.41"
libp2p-identity = "0.2"
libp2p-noise = "0.44"
libp2p-tls = "0.3"
libp2p-tcp = "0.41"
libp2p-yamux = "0.45"
tokio = { version = "1.0", features = ["full"] }
futures = "0.3"
```

```
rustls = "0.22"

rand = "0.8"

thiserror = "1.0"
```

결론

이 예제들은 Rust libp2p의 보안 계층 구현을 실제 코드를 통해 보여줍니다:

1. **Noise 프로토콜**: XX 핸드셰이크를 통한 상호 인증
2. **TLS 지원**: 표준 TLS 1.3 기반 보안
3. **키 관리**: Ed25519, RSA, ECDSA 등 다양한 키 타입 지원
4. **피어 인증**: 암호학적 신원 확인 과정
5. **Transport 통합**: 완전한 보안 네트워크 스택 구성

이러한 예제를 통해 libp2p의 보안 메커니즘을 이해하고 실제 애플리케이션에 적용할 수 있습니다.